CSE 753 – Advanced Reinforcement Learning
Study Notes: Lecture 09, Reward Learning

This note covers Lecture 09 (Reward Learning: goal classifiers, learning reward from human preferences, RLHF, DPO, and RLAIF). It is built from the lecture slides, the exam-focus guide, and the "what is expected" guide. Each concept is explained twice: once in plain, everyday language, and once with the precise technical statement a grader expects. Every equation is broken into a symbol table. Places where the slides contain a typo or an ambiguity are flagged explicitly, because the exam is open-book and rewards students who notice these and state a clean assumption.

The one idea that ties this whole lecture together: whenever we replace the true objective with a learned stand-in (a classifier, a reward model), the optimizer will chase the places where the stand-in is *wrong*, not just the places where it is right. Every failure mode in this lecture – classifier exploitation, reward hacking under RLHF – is this same idea wearing a different costume. Say this explicitly on the exam whenever it is relevant; it is repeatedly flagged as a full-marks signal.

1 Where Does the Reward Come From?

1.1 What is the concept?

In every RL problem we assume a reward function exists and tells the agent how good each state or action is. This slide asks the question underneath that assumption: in a real task (a robot, a dialogue agent, a self-driving car), who actually writes down that reward function?

1.2 Plain-English explanation

Think about training a puppy. In a simple game you can say "+1 point for every ball fetched." But how do you give a robot a "score" for making a good cup of tea, or give a chatbot a "score" for giving a helpful answer? Nobody can write a formula for that. In real tasks we usually only have a hand-made stand-in (a *proxy*) for what we actually want, because the true goal is too fuzzy to write as a formula.

1.3 Why not just imitate an expert instead?

The slide raises direct imitation learning (copy what an expert did) as one alternative, and lists why it is not enough by itself:

- Imitation only mimics actions; it never reasons about *why* those actions were good, so it cannot generalize to a new situation the expert never faced.
- The expert might move in ways the robot physically cannot (different "degrees of freedom"), so copying literally does not transfer.

- Sometimes there is no expert at all to record demonstrations from.

This motivates the rest of the lecture: instead of copying an expert, or hand writing a reward, can we *learn* a reward function from weaker, easier signals such as example goal states or human comparisons?

1.4 Problem it solves

It solves the "reward specification" problem: how to get a usable reward signal for tasks where no formula for the reward exists.

1.5 Problem it cannot solve (yet)

Just raising the question does not solve it. This slide is purely diagnostic; the rest of the lecture is the set of concrete fixes (goal classifiers, preference learning, RLHF, RLAIF).

2 Goal Classifiers

2.1 What is the concept?

A goal classifier is a small binary (yes/no) classifier trained to recognize "did the task succeed in this state?" Its output is then *used as the reward signal* for an RL algorithm.

2.2 Plain-English explanation

Suppose the task is "put the pencil case behind the notebook." Instead of writing a mathematical rule for success, you show the computer a pile of pictures where the task succeeded (label 1) and a pile where it did not (label 0), and you train an ordinary image classifier on those pictures. Now, whenever the robot ends up in some new state, you ask the classifier "does this look like success?" and whatever number it spits out (between 0 and 1) becomes the reward for that state.

2.3 The algorithm, step by step (slide 3)

1. Collect example states inside the goal set G (success) and outside it (failure).
2. Train a binary classifier $C_\phi(s)$ with inputs s_i and labels $\mathbf{1}(s_i \in G)$.
3. Run RL using the classifier's output as the reward.

Equation / notation table

Symbol	What it means
s_i	One example state (a snapshot of the world, e.g. a photo of the table).
G	The set of all states that count as "the task is done."
$\mathbf{1}(s_i \in G)$	A label that is 1 if state s_i is a success and 0 otherwise – this is exactly what you would type into a spreadsheet next to each photo.
$C_\phi(s)$	The trained classifier, with learnable parameters ϕ; given a state it outputs a number between 0 and 1.

What the whole idea is saying: "train an ordinary yes/no classifier on labelled snapshots, and reuse its confidence score as if it were a reward."

2.4 What can go wrong (slide 4) – Attribute the failure

Name of the phenomenon: the RL agent *exploits the classifier's weaknesses.*

Mechanism, step by step:

1. The RL agent is not told "reach success states"; it is told "maximize the classifier's output number."
2. Those are only the same thing on states the classifier was actually trained on.
3. RL exploration is very good at finding unusual, never-before-seen states.
4. On those unseen states the classifier's output is meaningless (it is guessing), and RL will happily steer toward whichever unseen state happens to score high, even though it is not a real success.

Canonical example (from the exam-focus guide): a cup left upside down on a coaster scores 0.91 from the classifier even though the task was never completed – the classifier had never seen an upside-down cup during training, so it guessed, and it guessed wrong in a way that happened to reward the agent.

2.5 The proposed fix (slides 5–6): retrain on visited states

Idea: every state the RL agent visits during rollouts is added to the *negative* set D_-, the classifier is retrained on a freshly rebalanced 50/50 mix of D_+ and D_-, and the loop repeats.

Symbol	What it means
D_+	The growing set of states known to be successes (goal states).
D_-	The growing set of states known to be failures, continually topped up with every state the RL agent has ever visited.
s_t	A state visited at time t during a rollout of the current policy π.
$D_- \leftarrow D_- \cup \{s_t\}$	"Add this newly visited state to the failure pile," regardless of whether it happens to actually be a success.

Plain-English reading of the loop: every time the agent finds a new "loophole" state that fools the classifier, that exact state gets thrown into the failure pile and the classifier is retrained. The classifier can never be fooled twice by the same trick, because the trick becomes a labelled negative example the moment it is used.

2.6 The paradox and its resolution – Justify the design (E4)

The worry: genuine goal states, once visited during rollouts, also get added to D_-. Doesn't that teach the classifier that the goal is a failure?

The resolution: no, because of the 50/50 *rebalancing* step, not because the goal states "look different." Work through it:

- A true goal state can appear on *both* sides: in D_+ (from the original positive examples, or from being reached during RL and recorded as success) and in D_- (from the blanket rule that every visited state is added to the negatives).
- D_+ starts small; D_- grows huge because *every* visited state goes there, success or not.
- Balancing the training batch to 50/50 means each example in the small D_+ is effectively *upweighted* relative to each example in the huge D_-.
- For a state that appears on both sides, the upweighted positive evidence is at least as strong as the negative evidence, so the best classifier output for it is $p \geq 0.5$.
- A true failure state, by contrast, appears *only* in D_- and gets driven toward 0.

Why this matters for exam marks: an answer that says "it still works because the goal state looks different from other states" earns zero credit. The question is specifically testing whether you know it is the *balancing step* doing the work, not anything about the state's appearance.

2.7 Advantages

- No reward function has to be hand-engineered; only labelled example states are needed, which are usually far easier to collect than a formula.
- Works for tasks that are easy to *recognize* but hard to *describe* mathematically (e.g. "the room looks tidy").

2.8 Disadvantages

- Vulnerable to exploitation by the very optimizer it is meant to guide, unless the retraining loop (slides 5–6) is used.
- Binary only: it can say "success" or "not success" but cannot rank two different successes against each other (a perfectly folded shirt and a barely acceptable one score the same).
- The retraining loop adds engineering complexity (a classifier retraining step nested inside the RL loop) and still needs an initial, reasonably diverse D_+/D_- to start from.

2.9 What problem it solves

Turns a task that has no formula for success into a supervised-learning problem: label some states, train a classifier, use the classifier's confidence as reward.

2.10 What it cannot solve

It cannot express *degrees* of success – there is no way for a goal classifier to say "this outcome is better than that outcome," only "this is a success/failure." It also cannot originate the idea of the goal on its own; a human still has to supply the initial labelled examples.

2.11 What problem it gave rise to

Its binary, all-or-nothing nature is exactly the gap that motivates the next section: learning reward from *human preferences* (which trajectory is better?), because preferences give a graded signal instead of a yes/no stamp.

2.12 When to use it

Use a goal classifier when success is visually or perceptually easy to recognize but has no clean symbolic description (most manipulation and navigation tasks), and when you are able to run the retrain-on-visited-states loop to keep it honest.

2.13 When not to use it

Avoid it when you need to rank the *quality* of successes, not just detect success versus failure (e.g. comparing two different helpful chatbot answers) – use preference-based reward learning (Section 3) instead.

2.14 Counterfactual – what if a part were done differently?

If step 5 (adding visited states to D_-) were skipped, the classifier would never see the agent's newly-discovered exploits, and the "what can go wrong" failure of Section 2.4 would recur indefinitely – the agent would keep finding new blind spots forever. If the 50/50 rebalancing were dropped and the raw (highly imbalanced) D_+/D_- were used instead, the massive D_- set would swamp the training signal and the classifier could be pushed to output near-zero for genuine goal states, since a small number of positive labels for that state would no longer be able to outweigh a large number of negative labels for it – which reintroduces exactly the same problem the retraining loop was built to avoid.

3 Learning Reward from Human Preferences

3.1 What is the concept?

Instead of asking a human to grade a single trajectory with a number, ask a much easier question: "Given two trajectories, which one is better?" Then fit a reward function so that it assigns a higher score to whichever one the human preferred.

3.2 Plain-English explanation

It is much harder for a person to say "this trajectory is worth exactly 7.3 points" than to say "I like trajectory A more than trajectory B." Comparisons are a natural, easy judgment for humans; absolute scores are not. So the lecture builds a reward function purely from these easier, relative comparisons.

3.3 The reward-consistency requirement (slide 8)

If a human says τ_w (the winner) is better than τ_l (the loser), we want a reward function r_θ such that the total reward it assigns to τ_w is higher than the total reward it assigns to τ_l:

$$\sum_{(s,a)\in\tau_w} r_\theta(s,a) \;>\; \sum_{(s,a)\in\tau_l} r_\theta(s,a)$$

Error flag. The lecture slide (slide 8) actually writes the right-hand side as $\sum_{(s,a)\in\tau_w} l_\theta(s,a)$ – both the sum and the subscript on the summation are copied from the left-hand side by mistake. The corrected statement, matching the surrounding text and every later slide, is the one shown

above: the second sum ranges over τ_l (the losing trajectory), not τ_w. Write the corrected form on the exam and note in one sentence that the slide has a typo.

Symbol	What it means
τ_w	The trajectory the human judged to be the *winner* (better one).
τ_l	The trajectory the human judged to be the *loser* (worse one).
(s, a)	One state-action pair visited along a trajectory.
$r_\theta(s, a)$	The learned reward model's score for taking action a in state s, with learnable parameters θ.

Plain reading: "whatever reward function we learn, it had better give a higher total score to the trajectory the human liked more."

3.4 The Bradley–Terry preference model

To turn "τ_w should score higher" into something we can train with gradient descent, the lecture defines a *probability* of preference using the logistic sigmoid σ:

$$P(\tau_a \succ \tau_b) = \sigma\big(r_\theta(\tau_a) - r_\theta(\tau_b)\big)$$

and then trains θ by maximizing the log-probability of the human's actual choices:

$$\max_\theta \ \mathbb{E}_{\tau_w,\tau_l}\Big[\log \sigma\big(r_\theta(\tau_w) - r_\theta(\tau_l)\big)\Big]$$

Symbol	What it means
$\sigma(x) = \frac{1}{1+e^{-x}}$	The sigmoid function. Turns any real number into a value between 0 and 1, so it can be read as a probability.
$r_\theta(\tau_a) - r_\theta(\tau_b)$	How much more reward the model currently thinks trajectory a deserves compared to trajectory b.
$P(\tau_a \succ \tau_b)$	The model's guess at the probability that a human would say trajectory a is better than trajectory b.
$\mathbb{E}_{\tau_w,\tau_l}[\cdot]$	Average over many human-labelled comparison pairs in the dataset.
$\log \sigma(\cdot)$	The log-likelihood of the human's actual answer; maximizing this pushes the reward gap in the direction the human indicated.

Plain-English reading of the whole equation: "the bigger the reward gap the model assigns between the winner and the loser, the closer σ gets to 1, i.e. the more confidently the model agrees with the human; train the model to make this agreement as strong as possible, on average, across all the comparisons we have."

3.5 Worked numeric example (Execute – E1)

Given $r_\theta(\tau_w) = 4.0$ and $r_\theta(\tau_l) = 1.5$:

$$\Delta = r_\theta(\tau_w) - r_\theta(\tau_l) = 4.0 - 1.5 = 2.5$$
$$\sigma(2.5) = \frac{1}{1 + e^{-2.5}} \approx 0.924$$
$$\text{loss} = -\log(0.924) \approx 0.079$$

Reading the number (E2): a small loss like 0.079 means the reward model already ranks this pair the way the human did – there is not much left to learn from this particular comparison. A large loss would mean the model currently disagrees with the human and this pair carries a strong training signal.

3.6 The reward-learning training loop (slide 9)

1. From a dataset of trajectories, sample a batch of k trajectories and ask humans to rank them (for LLMs, all k trajectories answer the same prompt).
2. Compute $r_\theta(\tau_1), \ldots, r_\theta(\tau_k)$ under the current reward model.
3. For all $\binom{k}{2}$ pairs in the batch, compute the gradient of $\mathbb{E}_{\tau_w,\tau_l}[\log \sigma(r_\theta(\tau_w) - r_\theta(\tau_l))]$.
4. Update θ using the computed gradient.
5. Repeat from step 2.

Combinatorics (Execute): ranking k items produces $\binom{k}{2}$ pairwise comparisons. For $k = 6$: $\binom{6}{2} = \frac{6\times5}{2} = 15$ comparisons. This is why ranking a whole batch at once is more data-efficient than asking for one pairwise comparison at a time – a single ranking of 6 items is worth 15 individual comparisons for free.

3.7 Why preferences instead of absolute scores – Justify the design

- Relative comparisons are a far easier, more natural judgment for a human to make than an absolute numeric score.
- The reward model only needs to be *discriminative* (correctly order pairs), not *calibrated* (produce a score on any particular numeric scale) – a much weaker and easier requirement to satisfy.

3.8 Advantages

- Sidesteps the impossible task of hand-writing a numeric reward.
- Cheap, natural signal to collect from ordinary humans (or, later, from another AI – Section 6).
- Ranking a batch of k items yields $\binom{k}{2}$ comparisons "for free," so it is data-efficient.

3.9 Disadvantages

- Only captures the *difference* between two rewards – the absolute scale of the reward function is never pinned down by this loss (this same fact reappears, with larger consequences, in the DPO derivation in Section 5).
- Needs many human comparisons, which is still a real cost at scale, and motivated Section 6 (RLAIF) as a way to remove that cost.

3.10 What problem it solves

Produces a trainable, differentiable reward function from human judgments that never required a single absolute numeric score.

3.11 What it cannot solve

By itself, this only trains a reward model; it does not yet say how to use that reward model to actually improve a policy. That is the job of the RLHF pipeline in the next sections.

3.12 What problem it gave rise to

Once you have a learned reward model instead of a true reward, you have created a new failure mode: the policy can be pushed to *over-optimize* the learned model rather than the real objective it stands in for. This is the reward-hacking problem addressed in Section 5.

3.13 Counterfactual

If the loss only used $r_\theta(\tau_w)$ alone (with no comparison to $r_\theta(\tau_l)$), the model would have no notion of relative quality at all – it would degenerate into trying to make every trajectory's reward large, which is not a meaningful training signal, since there would be nothing to push the reward of bad trajectories down.

4 RLHF: The Three-Stage Pipeline for LLMs

4.1 What is the concept?

For large language models, reward learning from preferences is embedded in a larger three-stage recipe: pre-train, then supervised fine-tune, then apply reinforcement learning against a learned reward model. This is the standard "RLHF" pipeline.

Stage	**Objective**	**Data / signal**
1. Large-scale pre-training	Next-token prediction	Massive, mixed-quality, unlabelled text corpus
2. Supervised fine-tuning	Maximum likelihood on demonstrations	Curated, higher-quality (prompt, response) pairs
3. RL from human feedback	Maximize the learned reward model minus a KL penalty to the pre-trained model	Human *pairwise preference comparisons* (x, y_w, y_l), used to train the reward model

Error flag. On the lecture slide, stage 3's subtitle literally repeats stage 2's subtitle ("on higher-quality (prompt, response)"). This is a copy-paste typo. Stage 3's actual data is *preference comparisons*, not plain (prompt, response) pairs – do not copy the slide's subtitle verbatim on the exam.

4.2 Plain-English explanation of the pipeline

1. **Pre-training:** the model first reads huge amounts of text and just learns to predict the next word. It becomes fluent, but it has not yet learned to be a good assistant.

2. **Supervised fine-tuning:** the model is shown a smaller set of high-quality example conversations and trained to imitate them directly, the same way ordinary supervised learning works.

3. **RLHF:** finally, the model generates several candidate answers to a prompt, a reward model (trained from human preference comparisons, as in Section 3) scores them, and reinforcement learning nudges the model's parameters toward answers that score higher.

4.3 Gathering preference data for LLMs (slides 10–11)

For a prompt x (e.g. "How do I cook an omelette?"), the model samples two candidate replies y and y'. A human is shown both and picks the better one, written $y \succ y'$. A reward model $r(x, y)$ (or $RM_\phi(x, y)$) is then trained on many such comparisons, exactly following the Bradley–Terry loss from Section 3.

4.4 Advantages of the three-stage pipeline

- Separates concerns cleanly: fluency (stage 1), instruction-following style (stage 2), and alignment with nuanced human judgment (stage 3).

- Reuses cheap, plentiful data (raw text) for the most expensive-to-train stage, and saves the costly human-labelled data for the stages that need it most.

4.5 Disadvantages

- Three separate training stages: expensive, slow, and each stage can introduce its own errors that propagate forward.

- Stage 3 needs a working reward model *and* an RL loop, both of which add engineering complexity and instability (see Section 5).

4.6 What problem it solves

Converts human notions of "a good response" – which have no formula – into a concrete training signal a language model can be optimized against, without needing millions of hand-written demonstrations for every possible prompt.

4.7 What it cannot solve

It does not, by itself, prevent the RL stage from over-optimizing the learned reward model (reward hacking) – see Section 5.

4.8 Counterfactual

If stage 3 skipped the reward model and instead reinforced the model directly against the raw preference labels (i.e. treated preference as the reward with no learned generalization), the model could only ever be evaluated on the exact pairs humans labelled – it would have no way to score a brand-new response the policy generates during RL, so RL could not proceed at all. The learned reward model is precisely what lets the signal generalize to unseen responses.

5 RLHF: Optimizing the Learned Reward Model

5.1 What is the concept?

Given a frozen reward model RM_ϕ, RLHF fine-tunes a copy of the language model, p_θ^{RL}, to produce responses that score highly under RM_ϕ, while adding a penalty that stops it from drifting too far from the original pre-trained model p^{PT}.

5.2 The objective (slide 12)

$$\mathbb{E}_{\hat{y}\sim p_\theta^{RL}(\hat{y}|x)}\left[RM_\phi(x,\hat{y}) - \beta \log\left(\frac{p_\theta^{RL}(\hat{y} \mid x)}{p^{PT}(\hat{y} \mid x)}\right)\right]$$

Symbol	What it means
$p^{PT}(\hat{y} \mid x)$	The frozen, pre-trained (or instruction-tuned) language model; the fixed "anchor" we do not want to drift too far from.
$p_\theta^{RL}(\hat{y} \mid x)$	The copy of the model currently being trained by RL; starts identical to p^{PT}, then its parameters θ are updated.
$\hat{y}$	A response generated by the current RL policy for prompt x.
$RM_\phi(x,\hat{y})$	The learned reward model's score for how good response $\hat{y}$ is for prompt x.
$\log\left(\frac{p_\theta^{RL}(\hat{y}\|x)}{p^{PT}(\hat{y}\|x)}\right)$	A per-response measure of how much more (or less) likely the RL model makes this response compared to the original pre-trained model – large when the RL model has drifted away from the original behaviour.
β	A single number (a "price tag") controlling how strongly drift away from p^{PT} is penalized.

Plain-English reading: "try to make the model's answers score highly on the learned reward model, but charge a penalty, priced at β, for every bit the model's behaviour drifts away from how it used to talk before RL started."

5.3 Reading the two extremes of β – Bound the claim (E5)

- $\beta \to 0$**:** the drift penalty disappears; the objective becomes pure reward maximization. The policy is then free to exploit any error in RM_ϕ, producing degenerate, high-scoring text that has lost fluency and general capability.

- $\beta \to \infty$**:** the penalty completely dominates the reward term, forcing $p_\theta^{RL} \approx p^{PT}$. The model stops changing at all – it is maximally safe, but it has learned nothing from the human preferences.

This is exactly the same shape of trade-off as the KL-constraint limits in actor-critic methods (Lecture 07): a constraint that is too loose lets the optimizer run wild against an imperfect target; a constraint that is too tight prevents any learning.

5.4 Reward hacking / over-optimization – Attribute the failure (E3)

Name of the phenomenon: reward hacking, also called over-optimization of a learned reward model.

Mechanism, step by step:

1. RM_ϕ was trained on a finite sample of human comparisons, so it is only accurate *on the distribution of responses it was trained on.*
2. The RL policy is explicitly searching for the responses that score highest under RM_ϕ.
3. That search process actively drives the policy toward unusual, off-distribution responses, precisely where RM_ϕ's accuracy has never been checked.
4. The result is a response that RM_ϕ rates very highly but that a human would actually judge as low quality, confidently wrong, or strange.

Diagnostic signs to look for (two together, not one alone):

1. The reward-model score keeps climbing while human-judged quality actually falls (confident but wrong outputs).
2. Outputs start looking qualitatively different from the pre-trained model's style, i.e. the KL divergence from p^{PT} has grown large, often with repetitive or degenerate stylistic tics.

Fix: increase β, and/or stop training earlier, and/or retrain the reward model on fresh, on-policy comparisons so it stays accurate on the region the policy currently occupies.

5.5 The shared-structure connection – worth stating explicitly

This is the same failure, in a new outfit, as classifier exploitation in Section 2: a learned stand-in for the true objective is only reliable on the data it was trained on, and an optimizer that searches hard enough will always find the stand-in's blind spots. On the exam, naming this connection explicitly (in one sentence) is called out as a signal of real understanding rather than rote recall.

5.6 Advantages

- Gives a tunable dial (β) that trades off "learn from preferences" against "stay safe and close to the original model."
- Works even though the true, ideal reward is unknown – it only needs a learned proxy.

5.7 Disadvantages

- Requires careful tuning of β; get it wrong in either direction and you get degenerate outputs or no learning at all.
- Needs a full RL training loop plus sampling from the policy at every step – computationally expensive and can be unstable.
- Vulnerable to reward hacking whenever β is too small or training runs too long.

5.8 What problem it solves

Gives a concrete, tunable way to fine-tune a language model against a learned notion of "good response" without completely destroying its original fluency and capabilities.

5.9 What it cannot solve

It cannot guarantee the learned reward model stays accurate forever as the policy moves; nothing in the objective itself detects when RM_ϕ has become unreliable – that has to be watched for externally (the two diagnostic signs above).

5.10 What problem it gave rise to

The expense and instability of running a full RL loop against a separately trained, frozen reward model is exactly what motivates Direct Preference Optimization (Section 5): can we skip the reward model and the RL loop entirely?

5.11 When to use it

Use full RLHF with an explicit reward model and RL loop when you want the flexibility to reuse the reward model for multiple downstream policies, or when online sampling during training is feasible and affordable.

5.12 When not to use it

Avoid it when training compute or stability is limited, or when a simpler, directly-supervised alternative (DPO) would reach a similar result more cheaply.

5.13 Counterfactual

If the KL penalty term were removed entirely and only $RM_\phi(x, \hat{y})$ were maximized, that is exactly the $\beta \to 0$ case above: reward hacking would set in quickly since nothing anchors the model to sensible, fluent language.

6 Direct Preference Optimization (DPO)

6.1 What is the concept?

DPO is a way to get the same effect as the full RLHF pipeline (fine-tune a language model to match human preferences) *without* training a separate reward model and *without* running an RL loop at all. It reduces the whole problem to a single supervised classification-style loss computed directly on preference pairs.

6.2 Plain-English explanation

The big insight is: the RLHF objective from Section 5 has an exact mathematical solution on paper – if you already knew the ideal reward model, you could write down the exact best-fine-tuned model in one formula, with no trial-and-error RL needed. DPO flips that formula around: instead of using it to compute the best model from a known reward, it uses it to say "whatever the reward model would have been, it can be written directly in terms of the model itself." That means the model becomes its own reward model, and the preference loss from Section 3 can be trained directly on the language model's parameters – no separate reward network, no RL loop, no sampling during training.

6.3 The derivation, one link at a time (examinable in full)

Step 1 – the closed-form optimum (slide 13). The RLHF objective from Section 5 has an exact best solution:

$$p^*(\hat{y} \mid x) = \frac{1}{Z(x)}\, p^{PT}(\hat{y} \mid x)\, \exp\!\left(\frac{1}{\beta} RM(x, \hat{y})\right)$$

Symbol	What it means
$p^*(\hat{y} \mid x)$	The exact, ideal fine-tuned model that would come out of solving the RLHF objective perfectly.
$Z(x)$	The normalizing constant (partition function) that makes $p^*(\cdot\|x)$ sum to 1 over all possible responses y; formally $Z(x) = \sum_y p^{PT}(y\|x) \exp(RM(x,y)/\beta)$. It depends only on the prompt x, not on any particular response.
$\exp\!\left(\frac{1}{\beta} RM(x, \hat{y})\right)$	Reshapes the pre-trained distribution to favour high-reward responses more strongly as β shrinks.

Plain reading: "the perfect fine-tuned model just takes the original model's probabilities and re-weights them to favour high-reward responses, renormalized so everything still sums to 1."

Step 2 – rearrange to express the reward as a log-ratio.

$$RM(x, \hat{y}) = \beta \log\left(\frac{p^*(\hat{y} \mid x)}{p^{PT}(\hat{y} \mid x)}\right) + \beta \log Z(x)$$

This is just algebra on Step 1: solve for $RM(x, \hat{y})$ instead of for $p^*(\hat{y}|x)$.

Step 3 – this holds for an arbitrary language model. Since the relationship in Step 2 is just algebra, it holds no matter which model plays the role of p^*. So we can plug in *any* candidate model p_θ^{RL} and *define* an implied reward model straight from it:

$$RM_\theta(x, \hat{y}) = \beta \log\left(\frac{p_\theta^{RL}(\hat{y} \mid x)}{p^{PT}(\hat{y} \mid x)}\right) + \beta \log Z(x)$$

Plain reading: "the policy is implicitly its own reward model" – once we know β, the pre-trained model, and the current policy, we already know what reward model would have produced this policy as the RLHF optimum. No separate reward network is needed.

Step 4 – substitute into the Bradley–Terry loss; $Z(x)$ cancels.

$$RM_\theta(x, y^w) - RM_\theta(x, y^l) = \beta \log \frac{p_\theta^{RL}(y^w \mid x)}{p^{PT}(y^w \mid x)} - \beta \log \frac{p_\theta^{RL}(y^l \mid x)}{p^{PT}(y^l \mid x)}$$

$$J_{\text{DPO}}(\theta) = -\mathbb{E}_{(x,y^w,y^l)\sim\mathcal{D}} \left[\log \sigma\Big(RM_\theta(x, y^w) - RM_\theta(x, y^l) \Big) \right]$$

Symbol	What it means
y^w, y^l	The human-preferred (winner) and non-preferred (loser) response to the same prompt x.
$\mathcal{D}$	The dataset of human preference triples (x, y^w, y^l).
$J_{\text{DPO}}(\theta)$	The final DPO training loss – an ordinary supervised loss, computed directly on the language model's own output probabilities, with no RL loop and no separate reward network.

6.4 Why $Z(x)$ cancels – Justify the design (E4)

What $Z(x)$ is: the partition function for prompt x: a sum over the *entire space* of possible responses y. That space is exponentially large (every possible sequence of tokens), so $Z(x)$ is intractable to compute exactly.

Why it cancels: $Z(x)$ depends only on the prompt x, not on the particular response y. The DPO loss only ever needs the *difference* between the implied reward of two responses to the *same* prompt x (y^w and y^l). Since $\beta \log Z(x)$ is added identically to both $RM_\theta(x, y^w)$ and $RM_\theta(x, y^l)$, it is subtracted from itself in the difference and disappears – without ever having to be computed.

The practical consequence, the part most often missed: because only the difference survives, DPO training can compute *relative* rewards between two responses to the same prompt exactly, but the *absolute* reward of a single response is only known up to the unknown constant $\beta \log Z(x)$. That is completely sufficient for preference optimization (which only ever compares pairs), but it is *not* sufficient for any downstream use that needs a calibrated, absolute reward score for a single response on its own.

6.5 Worked numeric example (Execute – E1)

Given $\beta = 0.5$ and log-ratio values β-scaled to 1.2 (for y^w) and 0.2 (for y^l):

$$\text{argument} = 0.5 \times (1.2 - 0.2) = 0.5$$
$$\sigma(0.5) = \frac{1}{1 + e^{-0.5}} \approx 0.622$$
$$J_{\text{DPO}} = -\log(0.622) \approx 0.475$$

Reading the number: the loss is moderate-sized, meaning the model currently favours y^w over y^l by only a small margin – there is meaningful training signal left in this pair, unlike the earlier Bradley–Terry example where the reward gap was already large and confident.

6.6 DPO's advantage over full RLHF, in one sentence

DPO removes the separate reward-model training stage and the entire RL sampling loop, turning preference alignment into a single supervised, classification-style loss computed directly on log-probabilities – with no reward hacking of a separate frozen reward model, and generally more stable and cheaper to train.

6.7 Advantages

- No separate reward network to train, store, or maintain.
- No RL loop and no on-policy sampling during training – ordinary supervised-style optimization.
- Cannot hack a reward model that does not exist as a separate object.

6.8 Disadvantages

- Only ever learns *relative* preferences between pairs; it never produces a calibrated absolute reward for a single response.
- Still fundamentally needs human (or AI) preference-pair data – it does not remove the labelling requirement, only the reward-model and RL-loop machinery built on top of it.
- Because there is no explicit reward model, tools that rely on an explicit, standalone reward signal (e.g. for reward shaping or auditing) are not directly available.

6.9 What problem it solves

Removes the cost, complexity, and reward-hacking risk of the separate reward model plus RL loop in standard RLHF, while producing (in principle) the same optimal fine-tuned model.

6.10 What it cannot solve

Because absolute reward is unrecoverable (only known up to $\beta \log Z(x)$), DPO cannot supply a calibrated reward score for any single response in isolation – if a downstream system genuinely needs an absolute quality score (not just better-or-worse comparisons), DPO is the wrong tool and an explicit reward model is still required.

6.11 What problem it gave rise to

DPO still depends on a fixed, pre-collected preference dataset; it has no built-in mechanism to acquire fresh preference labels for new, off-distribution responses the way an online RLHF loop with a live reward model can. If the policy drifts far from the data the preference pairs were collected on, DPO has no online correction (unlike RLHF, which the note on slide 9 mentions can be run inside the loop of online RL).

6.12 When to use it

Use DPO when a fixed preference dataset is already available and the goal is a stable, cheap, single-stage fine-tune, with no need for an explicit, standalone reward model afterward.

6.13 When not to use it

Avoid DPO when the application specifically needs an absolute, calibrated reward score (e.g. to combine with other scoring systems, or for reward shaping in a downstream RL problem), or when preference data needs to be collected continually, on-policy, as training proceeds.

6.14 Counterfactual – what if a part were done differently?

If Step 3 had substituted p_θ^{RL} for a response to a *different* prompt x' instead of the same prompt x, the $Z(x)$ terms would not match ($\beta \log Z(x) \neq \beta \log Z(x')$) and would not cancel – this is precisely why DPO's preference pairs must always compare two responses to the *same* prompt.

7 Learning Reward from AI Feedback (RLAIF / Constitutional RL)

7.1 What is the concept?

Instead of asking a human "which response is less harmful?", ask another language model the same question, and use its answer in place of a human preference label everywhere in the pipeline above (Sections 3–5).

7.2 The key insight (slide 15), quoted as the exam wants it

"Critique is easier than generation." A model that cannot reliably *produce* the best possible response can still reliably *judge* which of two given responses is less harmful. That asymmetry between generating and judging is the entire justification for letting an AI stand in for a human annotator.

7.3 Plain-English explanation

Imagine a student who cannot write a great essay from scratch, but who can still tell you, given two draft essays, which one reads better. Judging is often a much easier skill than creating. RLAIF leans on exactly this: even if a language model is not good enough to write the best possible answer, it can often be trusted to say "answer A is safer than answer B."

7.4 Advantages at scale

- Essentially unlimited throughput, at a tiny marginal cost compared to paying and scheduling human annotators.
- Consistent application of a single, written policy or "constitution," with no annotator-to-annotator variance or annotator fatigue over long sessions.
- Scales instantly to new content volumes.
- Avoids exposing human moderators to large volumes of harmful or disturbing content.

7.5 The unique risk – Bound the claim (E5)

The one risk that does not apply to human annotators: the AI annotator's own biases and blind spots are *systematic and correlated across every single label it produces.* Independent human annotators make noisy, largely independent mistakes that tend to average out across a large dataset. A single AI annotator model does not have that property – if it has a blind spot, that exact blind spot shows up identically in every label it touches. The reward model then inherits that bias wholesale, and the RL policy optimizes directly against it, with nothing independent left anywhere in the loop to catch the drift.

7.6 Result from the slides (slide 15, Pareto plot)

Constitutional RL (AI feedback with a written constitution) is shown tracing out a curve strictly above and to the right of standard RLHF on a plot of helpfulness versus harmlessness – described on the slide as a *Pareto improvement*: better on both axes at once, not merely a trade of one for the other.

7.7 Comparison table: goal classifiers vs. RLHF

Aspect	**Goal classifier**	**RLHF (human preferences)**
Type of human input	A set of labelled example states (success/failure), given once, up front	Ongoing pairwise comparisons between rollouts or responses
Handles nuanced quality?	No – binary success/failure only; cannot rank two successes against each other	Yes – relative preferences give a graded, continuous reward signal
Main failure mode	Agent exploits states the classifier was never trained on	Reward hacking / over-optimization of the learned reward model

The connection to notice: these are the *same underlying failure* (a learned proxy gets exploited by the optimizer built on top of it) showing up at two different scales of the same idea.

7.8 Advantages

- Removes the human-labelling bottleneck almost entirely.
- More consistent than a pool of human annotators.

7.9 Disadvantages

- Inherits and amplifies the annotator model's own systematic biases, correlated across the entire dataset.
- No independent ground truth left in the loop to detect when the annotator model itself has drifted or is wrong.

7.10 What problem it solves

Removes the cost and throughput bottleneck of collecting human preference labels at the scale needed for large models.

7.11 What it cannot solve

It cannot supply an independent check on the annotator's own judgment – if the annotator model is systematically biased or has a blind spot, RLAIF has no built-in way to detect or correct that, unlike a diverse pool of independent human annotators whose individual errors tend to cancel out.

7.12 What problem it gave rise to

The systematic-bias risk described above is a new, distinct failure mode that a purely human-labelled pipeline did not have to the same degree – it motivates ongoing research into diversifying or auditing AI annotators, and combining AI and human feedback rather than relying on either alone.

7.13 When to use it

Use RLAIF when human-labelling throughput is the bottleneck, when a clear written policy (a "constitution") can specify what "better" means, and when some risk of correlated annotator bias is acceptable or can be independently audited.

7.14 When not to use it

Avoid relying on RLAIF alone when independent ground truth or auditability is critical, or when the judging model itself is known to share the same blind spots you are trying to correct for in the policy being trained.

7.15 Counterfactual

If the AI annotator were replaced by a diverse panel of several different AI models (rather than one), the correlated-bias risk described above would be reduced, because different models are less likely to share the exact same blind spots – much closer to how independent human annotators' errors average out.

8 Summary: The Thread Running Through the Whole Lecture

Stage	**The recurring pattern**
Hand-designed reward	Impossible to write for most real tasks.
Goal classifier	A learnable stand-in, but the RL agent exploits states the classifier was never trained on.
Retrain on visited states	Removes the exploit, but only gives a binary success/failure signal.
Human preferences (Bradley–Terry)	Gives a graded signal instead of binary, but needs a full RLHF pipeline (reward model + RL loop) to use it.

RLHF with KL penalty	Works, but the RL policy can over-optimize ("reward-hack") the learned reward model, off its valid distribution.
DPO	Removes the separate reward model and RL loop entirely, but only ever recovers *relative*, not absolute, reward.
RLAIF	Removes the human-labelling bottleneck, but introduces a new, systematic (not averaging-out) annotator-bias risk.

The single sentence to remember: optimizing a learned proxy for an objective makes the optimizer seek out exactly the places where the proxy is wrong – this is the same phenomenon behind classifier exploitation, RLHF reward hacking, and (per the exam-focus guide) OOD value overestimation in offline RL from Lecture 08. Naming this connection, in one sentence, whenever a question touches any one of these three ideas is repeatedly flagged as the mark of real understanding rather than memorized recall.